



CONCORRÊNCIA E MULTITHREADING

AULA 03 - CONCURRENT COLLECTIONS



AGENDA

- ❑ Concurrent Collections: Estruturas de Dados para um Mundo Paralelo
- ❑ Por que ArrayList e HashMap não são suficientes?
- ❑ Alternativas seguras e performáticas para estado compartilhado.



A LIMITAÇÃO DAS COLEÇÕES SINCRONIZADAS

- ❑ `Collections.synchronizedMap(new HashMap<>())`: Cria um wrapper onde cada método (`get`, `put`, `remove`) é `synchronized`.
- ❑ Mecanismo de Locking: Usa um único lock para a coleção inteira (locking de granularidade grossa).
- ❑ Problema de Performance: Apenas uma thread pode acessar a coleção por vez, criando um gargalo de contenção e limitando severamente a escalabilidade.



CONCURRENTHASHMAP: A ESCOLHA PADRÃO PARA MAPAS CONCORRENTES

- ❑ Mecanismo de Locking Fino: Em vez de um lock global, bloqueia apenas partes do mapa (nós/buckets) durante as operações de escrita.
- ❑ Alta Concorrência: Permite que múltiplas threads leiam e escrevam simultaneamente em diferentes partes do mapa. Leituras são geralmente não-bloqueantes.
- ❑ Iteradores Seguros: Os iteradores são "fracamente consistentes" e nunca lançam `ConcurrentModificationException`.
- ❑ Restrição: Não permite chaves ou valores null.



COPYONWRITEARRAYLIST: OTIMIZADO PARA LEITURAS

- ❑ Estratégia "Copiar ao Escrever": Toda operação de modificação (add, set, remove) cria uma nova cópia do array subjacente.
- ❑ Leituras Rápidas e Seguras: As operações de leitura são muito rápidas, pois não exigem locks e operam em uma cópia imutável.
- ❑ Caso de Uso: Ideal para coleções que são lidas com muita frequência, mas modificadas raramente (ex: listas de listeners/observadores).
- ❑ Custo: Modificações são caras ($O(n)$) devido à cópia do array.



BLOCKINGQUEUE: O PADRÃO PRODUTOR-CONSUMIDOR

- ❑ Interface BlockingQueue: Uma fila que bloqueia operações quando necessário.
- ❑ put(e): Bloqueia se a fila estiver cheia, esperando por espaço.
- ❑ take(): Bloqueia se a fila estiver vazia, esperando por um item.
- ❑ Fundação do Padrão Produtor-Consumidor: Desacopla threads que produzem dados daquelas que os consomem, gerenciando o fluxo de forma segura.



ATÉ A PRÓXIMA!